

第 30 讲练习题与答案: nn.Module

练习题

1. 用一句话分别说明 `__init__()` 和 `forward()` 的职责。
2. 对 `nn.Linear(5,7)`, 写出 `weight.shape`、`bias.shape` 和参数总数。
3. 一个 $4 \rightarrow 8 \rightarrow 3$ 的两层 MLP, 总参数量是多少?
4. 为什么 `self.x = torch.tensor(...)` 不会自动出现在 `model.parameters()` 中?
5. `Parameter` 与 `Buffer` 的核心区别是什么? 二者都可能进入 `state_dict()` 吗?
6. 为什么保存了 `state_dict()`, 加载时通常仍需要先创建同结构的模型对象?
7. 为什么日常推荐 `model(x)`, 而不是直接 `model.forward(x)`?
8. 若某个 `Parameter` 出现在 `named_parameters()` 中, 但执行 `backward` 后它的 `grad` 仍是 `None`, 至少列出两个可能原因。
9. 判断: `model.to('cuda')` 一定会把所有挂在 `self` 上的 `Tensor` 属性移动到 GPU。对还是错?
10. LLM 的一个 Transformer Block 如果写成一个 `nn.Module`, 你读代码时最先应该看哪两个位置? 为什么?

答案

1. `__init__()` 负责定义并注册模型组件/状态; `forward()` 负责定义输入 Tensor 到输出 Tensor 的实际计算路径。
2. `weight.shape=(7,5)`; `bias.shape=(7,)`; 参数总数 $7 \times 5 + 7 = 42$ 。
3. 第一层: $4 \times 8 + 8 = 40$; 第二层: $8 \times 3 + 3 = 27$; 总计 67。
4. 因为普通 Tensor 属性没有 `nn.Parameter` 的 Module 参数注册语义。若需要可训练, 应使用 `Parameter` 或由子 Module 内部注册。
5. `Parameter` 通常由优化器训练; `Buffer` 不由优化器训练, 但属于模型持久状态。默认持久 `Buffer` 与 `Parameter` 都进入 `state_dict()`。
6. 因为 `state_dict()` 主要保存权重/Buffer 的数值状态, 并不等于 Python 类定义和完整 Forward 代码。加载时需要有能接收这些键和 Shape 的模型结构。
7. `model(x)` 会经过 `nn.Module.__call__` 的通用机制, 再调用 `forward()`, 例如 `hooks` 等机制依赖这条路径。
8. 可能包括: 该 `Parameter` 没有参与当前 Loss 的计算图; 某条路径被 `detach/no_grad` 切断; `Parameter` 的 `requires_grad=False`; 尚未执行 `backward`; 代码中实际使用了另一个 Tensor 而不是这个 `Parameter`。
9. 错。Module 会递归管理已注册的 `Parameter/Buffer/子 Module`; 普通 Tensor 属性不应假设会得到同样的管理语义。
10. 先看 `__init__()`, 确认 `Attention`、`Norm`、`MLP` 等组件及 `Parameter`; 再看 `forward()`, 追踪 `Residual`、`Mask`、`Shape` 和数据流。两者合起来才能知道“有哪些参数”和“这些参数怎样参与计算”。

科研代码审查时, 不要只看到“模型 class 写得很复杂”就默认所有参数都被训练。检查 `named_parameters()`、实际 Forward 路径和梯度, 是最低成本的验证手段。